Clase 04: Memoria y Caché (cont.)

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Martes 17 de Marzo de 2026

Repaso: Memoria Caché

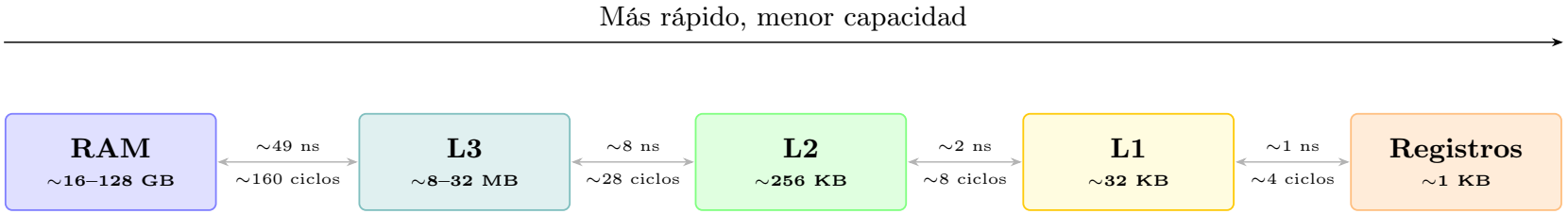
Mapeo y Políticas de Caché

Caché y Multicore

Ejemplos prácticos

Jerarquía de memoria y latencia

Cada nivel es más rápido pero más pequeño:

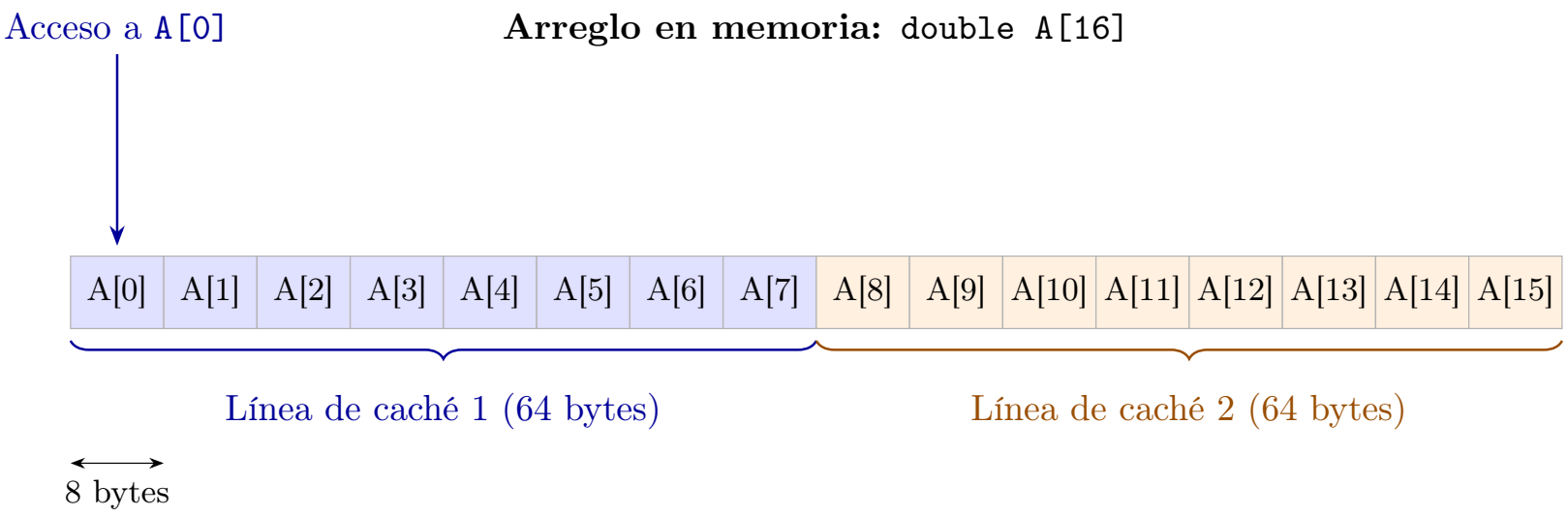


Ejemplo de latencias típicas (procesador a 3,5 GHz):

Nivel	Latencia (ciclos)	Latencia (*ns*)
L1 cache	~ 4	~ 1
L2 cache	~ 12	~ 3
L3 cache	~ 40	~ 11
RAM	~ 200	~ 60

Línea de caché

Los datos no se mueven byte a byte, sino en bloques de 64 bytes llamados **líneas de caché**:



- Al acceder a `A[0]`, el hardware carga la línea completa: `A[0]` a `A[7]`.
- Los accesos siguientes a `A[1]`, ..., `A[7]` son inmediatos (ya están en caché).
- Los movimientos caché-memoria los gestiona el hardware automáticamente.

Hit, miss y tipos de miss

- *Cache hit* : el dato solicitado ya está en caché.
- *Cache miss* : el dato no está en caché; hay que traerlo de un nivel inferior.

Tipos de miss:

- **Compulsory**: dato nunca cargado antes.
- **Capacity**: caché lleno; hay que desalojar algo.
- **Conflict**: dos datos necesarios se mapean al mismo slot .
- **Invalidation** (multicore): otro núcleo modificó el dato.

Localidad espacial y temporal

Los programas eficientes explotan dos tipos de **localidad**:

- **Localidad temporal**: un dato accedido recientemente se accederá de nuevo pronto.
  Ejemplo: variable de un bucle, acumulador.

- **Localidad espacial**: si se accede a una dirección, pronto se accederán direcciones cercanas.
  Ejemplo: recorrido secuencial de un arreglo.

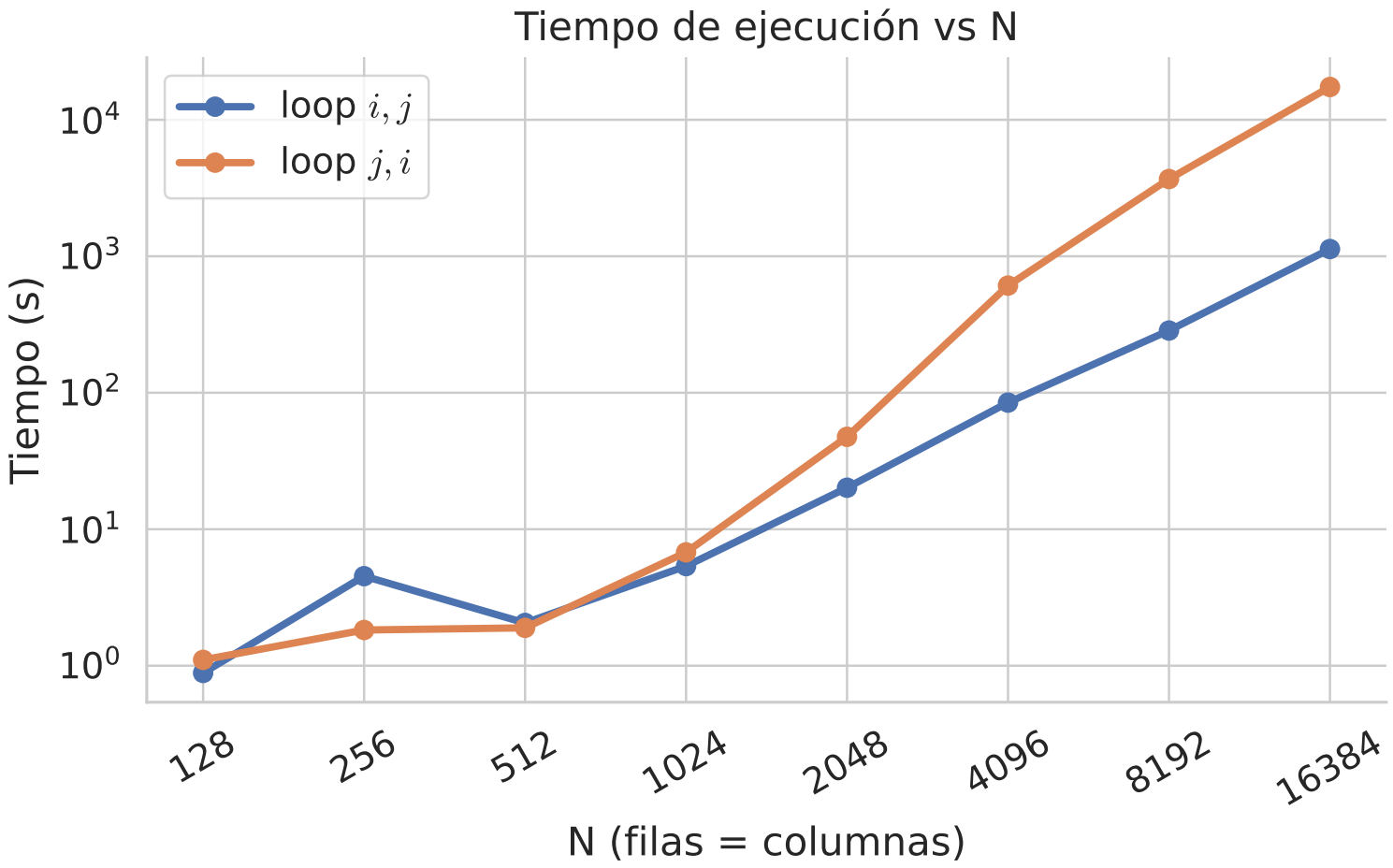
Ejemplo: orden de loops e impacto en caché

En C/C++, las matrices se almacenan por **filas** (*row-major*):

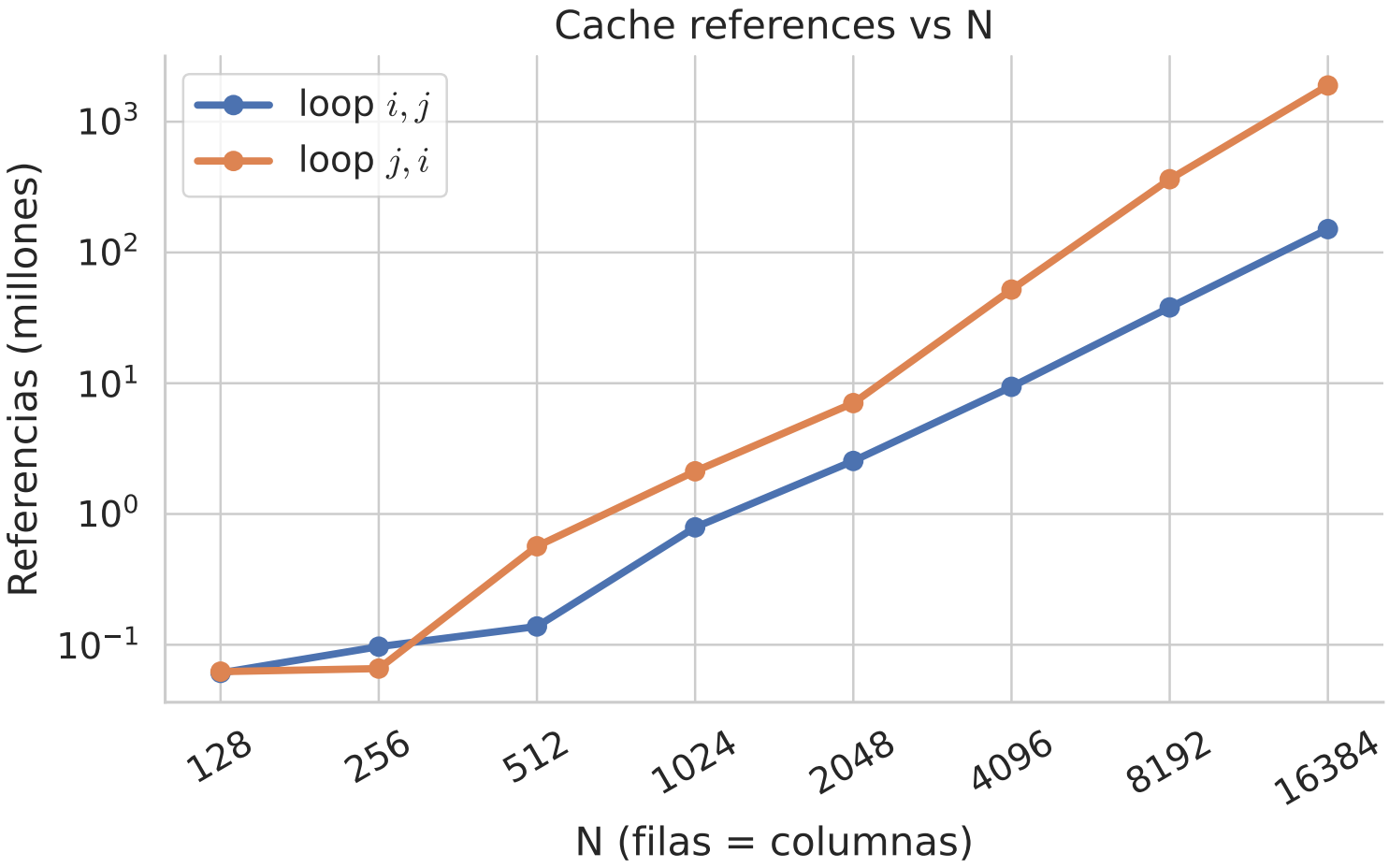
```
const int N = 4096;
std::vector<double> A(N * N); // memoria contigua

// Loop i,j: j varia rapido -> acceso contiguo -> cache hits
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    sum += A[i*N + j];

// Loop j,i: i varia rapido -> saltos de N*8 bytes -> cache
    misses
for (int j = 0; j < N; j++)
  for (int i = 0; i < N; i++)
    sum += A[i*N + j];
```

- Con $N = 4096$ y `double` (8B), cada fila ocupa 32 KiB.
- El loop j, i salta una fila completa entre accesos: produce muchos cache misses.

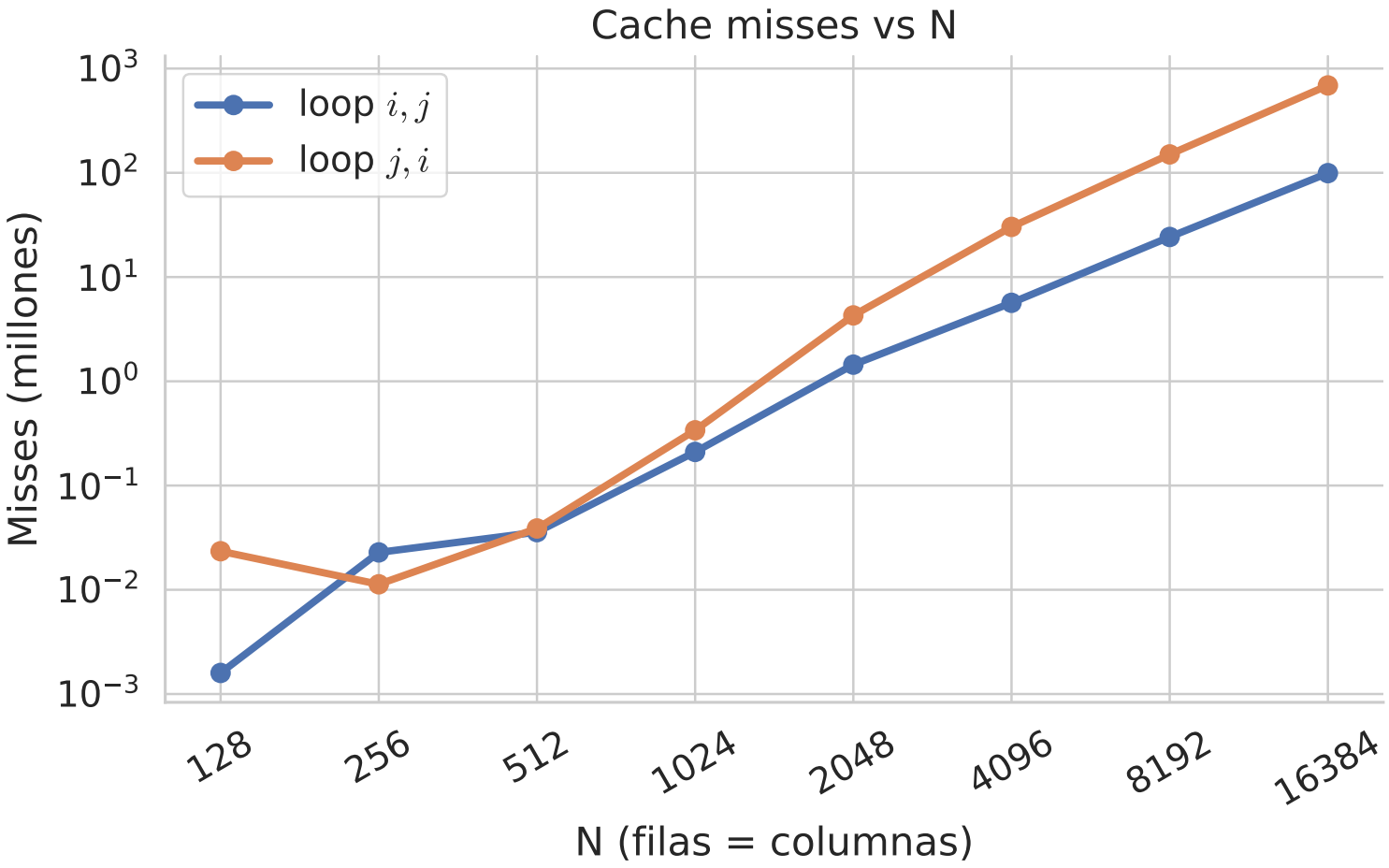
Loop i, j vs j, i: tiempo de ejecución



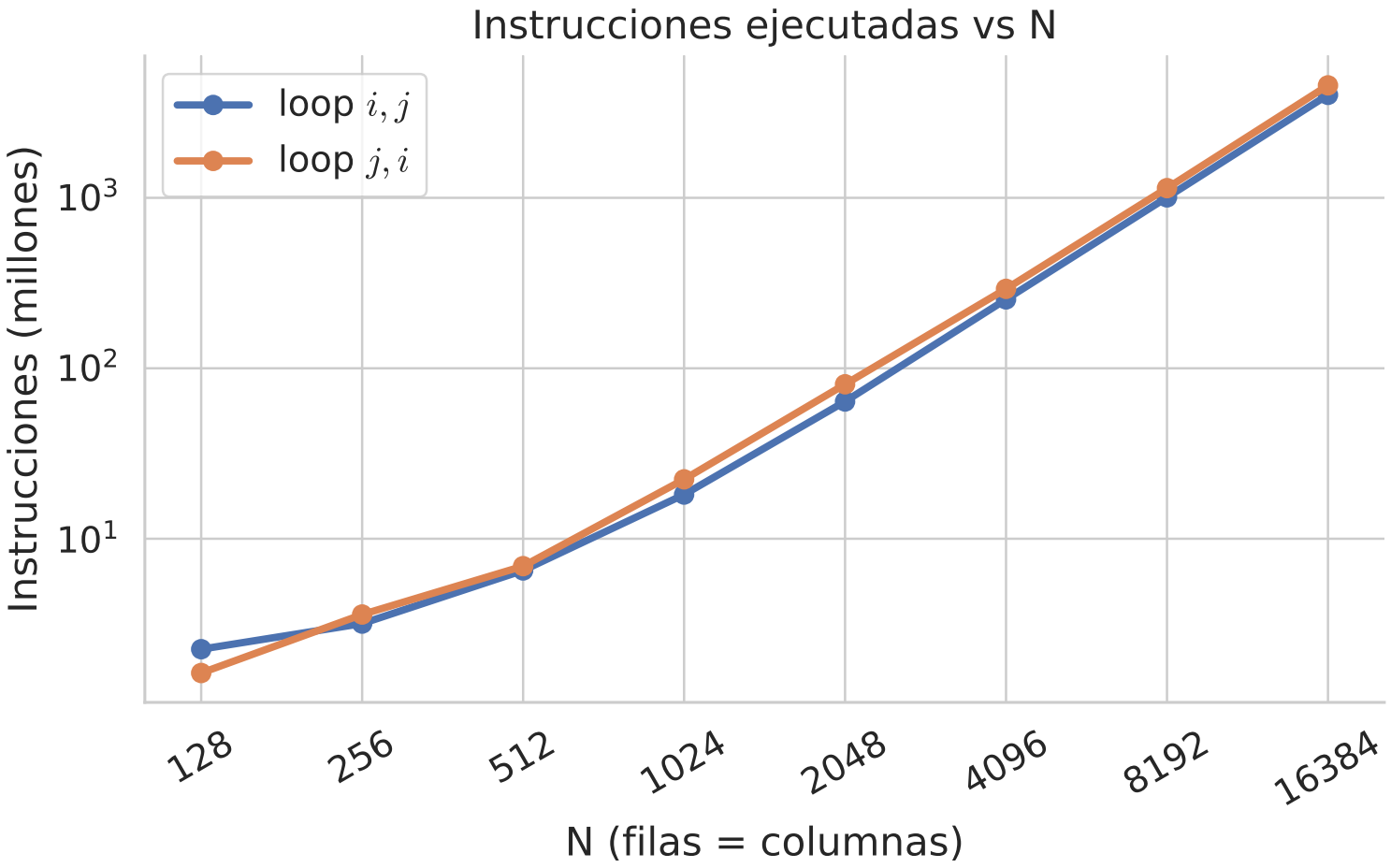
Loop i, j vs j, i: cache references



Loop i, j vs j, i: cache misses



Loop i, j vs j, i: instrucciones



Mapeo de caché: problema

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- ¿Cómo se determina en qué lugar de la memoria caché almacenaremos un determinado dato proveniente de la memoria RAM?
- No es posible asignar a cada dato un lugar distinto, porque RAM es mayor que Caché.
- Necesitamos un **mapeo** que, según de dónde viene el dato, asigne un lugar en la memoria caché.
- Este mapeo forma parte del **hardware**.

Direcciones de memoria

Cada byte en memoria tiene una **dirección** única, representada en binario.

¿Cuántos bits necesitamos?

- Con n bits podemos distinguir 2^n direcciones distintas.
- RAM de 4 GiB $= 2^{32}$ bytes. Necesitamos **32 bits**.
- Caché de 64 KiB $= 2^{16}$ bytes. Necesitamos **16 bits**.

Una dirección de memoria es simplemente el número identificador de un byte.

Mapeo directo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM (32 bits) $\longrightarrow$ Dirección Caché (16 bits)

- **Mapeo directo:** tomamos los últimos 16 bits de la dirección.
- Es rápido y sencillo (solo un *and* de bits).
- Pero distintas direcciones de RAM que coincidan en sus últimos 16 bits se mapean al mismo lugar en caché.
- **Principio del palomar:** 2^{32} direcciones de RAM deben caber en 2^{16} posiciones de caché. Inevitablemente, múltiples direcciones comparten el mismo slot.

Mapeo directo: ejemplo

Supongamos un caché L1 de 64 KiB, con direcciones de 16 bits.

```
1     double A[3][8192];
2     for (i=0; i<8192; i++)
3       a[2][i] = ( a[0][i]+a[1][i] )/2.;
```

- Cada fila ocupa $8192 \times 8 = 64$ KiB: exactamente el tamaño del caché.
- Las direcciones de `a[0][i]`, `a[1][i]` y `a[2][i]` están separadas por exactamente 64 KiB $= 2^{16}$ bytes.
- Por lo tanto, sus últimos 16 bits coinciden: el mapeo directo las lleva al mismo slot.

Mapeo directo: ¿qué ocurre en el ciclo?

```
a[2][i] = ( a[0][i] + a[1][i] ) / 2.0
```

En cada iteración, los tres acceden al mismo slot del caché:

1. Cargar `a[0][i]` de RAM a caché: *miss*. Llevar a registro r_0.
2. Cargar `a[1][i]` de RAM a caché (mismo slot): *miss*, **desaloja** `a[0][i]`. Llevar a registro r_1.
3. Sumar $r_0 + r_1$ y dividir por 2 en registros. Resultado en r_2.
4. Escribir r_2 en `a[2][i]`: cargar en caché (mismo slot), *miss*, **desaloja** `a[1][i]`.
5. Siguiente iteración: `a[0][i+1]` fue traído junto con `a[0][i]` en el paso 1 (misma línea de caché), pero fue desalojado en el paso 2.

   Resultado: 3 *misses* por iteración.

Cache miss por conflicto

Este es un ejemplo de **conflict miss**: direcciones distintas que se mapean al mismo slot de caché.

- Al acceder alternadamente a datos que comparten slot, cada acceso desaloja al anterior.
- La caché puede tener espacio libre, pero el mapeo directo no permite usarlo.
- **Solución**: caché *set-associative*.

Completamente asociativo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- Caché asociativo: los datos pueden ir donde sea (en la memoria caché)
- A esto se le llama **completamente asociativo**.
- Lamentablemente, es costoso de construir.

K-asociativo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- Caché asociativo: los datos pueden ir a más de un lugar (en la memoria caché)
- A esto se le llama **K-asociativo**.
- $K \geq 2$.
- En Linux: `getconf -a | grep CACHE` y `lscpu | grep cache`
- En macOS: `sysctl -a | grep cache`

K-asociativo: ejemplo

¿Qué ocurre con el mismo código con un caché **2-asociativo**?

```
1        double A[3][8192];
2        for (i=0; i<8192; i++)
3          a[2][i] = ( a[0][i]+a[1][i] )/2.;
```

- El caché sigue siendo 64 KiB, pero ahora tiene 4096 sets de 2 vías cada uno.
- Las filas siguen separadas por 64 KiB, pero el periodo de aliasing es ahora 2^{15} bytes (la mitad): `a[0][i]` y `a[1][i]` mapean al mismo set.
- Con 2 vías, `a[0][i]` y `a[1][i]` caben simultáneamente en el set.
- Pero `a[2][i]` es un tercero en el mismo set: desaloja a uno de los dos anteriores.
- Resultado: aún hay *conflict misses*, pero menos que con mapeo directo.

Políticas de reemplazo

Cuando la caché está llena y llega un dato nuevo, ¿qué línea se desaloja?

- **LRU** (*Least Recently Used*): se desaloja la línea que lleva más tiempo sin ser accedida.
  Buena aproximación, pero costosa de implementar en hardware para alta asociatividad.

- **FIFO** (*First In, First Out*): se desaloja la línea más antigua, sin importar cuántas veces fue accedida.

- **Random**: se elige una línea al azar. Sorprendentemente competitivo y muy barato de implementar.

En la práctica, los procesadores modernos usan variantes de **pseudo-LRU** (menos costoso que LRU puro).

Políticas de escritura

Cuando el procesador modifica un dato en caché, ¿cuándo se actualiza la memoria principal?

- ***Write-through***: cada escritura en caché se propaga inmediatamente a memoria.
  Coherencia simple, pero alto tráfico al bus de memoria.

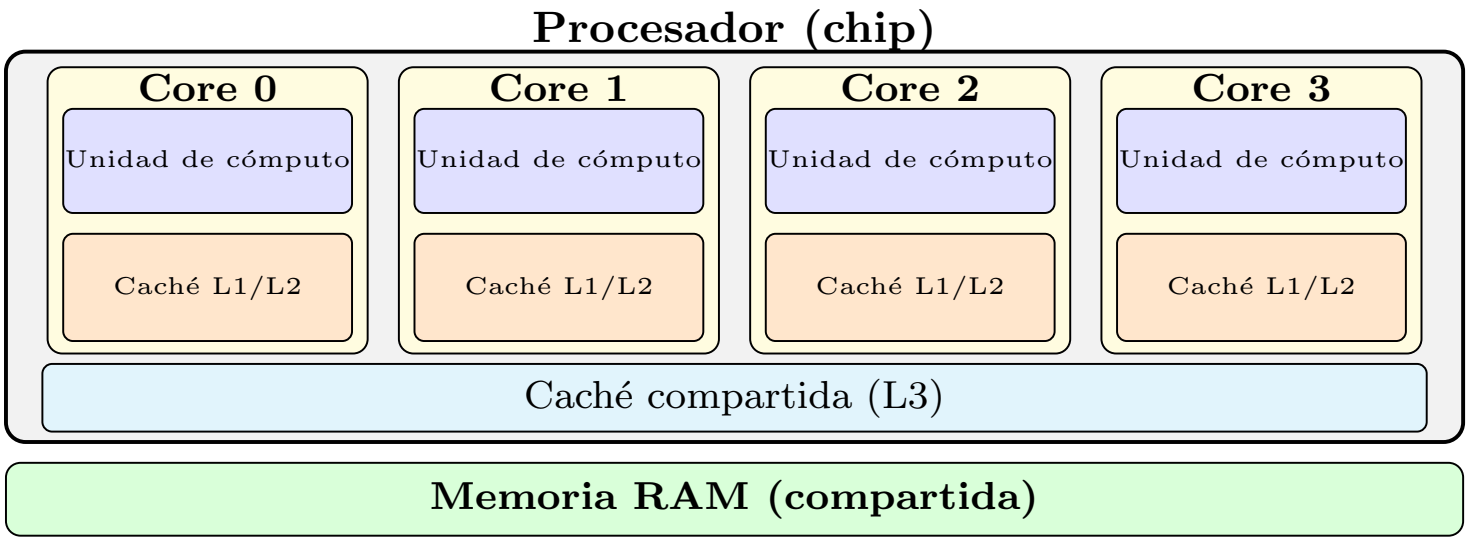
- ***Write-back***: la escritura se hace solo en caché; la línea se marca como *dirty* y se escribe a memoria solo al ser desalojada.
  Menos tráfico, pero coherencia más compleja en multicore.

La mayoría de los procesadores modernos usan ***write-back*** por eficiencia.

Estados de una línea de caché

- **Scratch**. La línea no contiene ningún dato válido.
- **Valid**. La línea es una copia correcta de memoria principal.
- **Dirty**. La línea ha sido modificada, pero aún no re-escrita en memoria.

Caché y multicore



Jerarquía de caché en multicore

- **L1 y L2** son privadas para cada núcleo: cada core tiene su propia copia, inaccesible directamente por los demás.
- **L3** es compartida entre todos los núcleos del procesador: todos leen y escriben en el mismo banco de caché.
- Datos en L1/L2 son más rápidos pero no visibles entre núcleos.

  Datos en L3 son visibles por todos, pero con mayor latencia.

Caché en sistemas multicore

En un procesador multicore, cada núcleo tiene su propia caché L1 (y a veces L2).

- Cada núcleo puede leer y escribir datos de forma independiente.
- La memoria RAM es **compartida** entre todos los núcleos.
- Si dos núcleos tienen copias del mismo dato en sus cachés locales y uno lo modifica, el otro tiene una copia desactualizada.
- Esto introduce un nuevo problema: ¿cómo mantener la **consistencia** entre las cachés?

Coherencia

- Existe el potencial de que ocurra un conflicto si más de un procesador tiene una copia de un mismo dato.
- **Coherencia caché**.
  Asegurarse que todos los datos en caché sean copias precisas de la memoria principal.
- Si un procesador modifica, el otro procesador debe actualizar su copia.
- Pueden existir niveles de caché **privados** para un procesador.

Estados respecto a un dato

Modelo MSI

Un modelo más simple es MSI:

- **Modified**. Ha sido modificado y necesita ser escrito.
- **Shared**. Presente en al menos un caché, no modificado.
- **Invalid**. No presente, o bien ha sido modificado en otro caché.

Operaciones en protocolos de coherencia

Las transiciones entre estados se producen por dos tipos de eventos:

Acciones del procesador local (no visibles en el bus):

- **PrRd**: el procesador lee un dato de su caché.
- **PrWr**: el procesador escribe un dato en su caché.

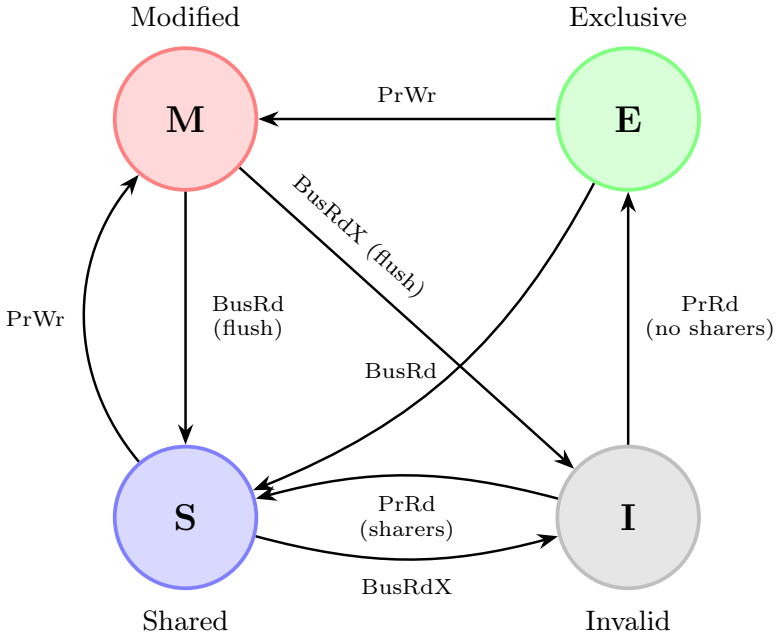
Transacciones en el bus (visibles para todos los núcleos):

- **BusRd**: un núcleo solicita leer un bloque; los demás observan la solicitud en el bus.
- **BusRdX**: un núcleo solicita leer un bloque *con intención de escribir*. Invalida todas las demás copias.

Cada núcleo *espía* (*snoops*) el bus: al detectar un BusRd o BusRdX, actualiza el estado de sus propias líneas de caché.

Modelo MESI

El protocolo **MESI** extiende MSI con el estado **Exclusive**:



- **Modified**: línea modificada localmente, no escrita a memoria.
- **Exclusive**: única copia, pero *limpia* (igual a memoria).
- **Shared**: presente en múltiples cachés, no modificada.
- **Invalid**: línea no presente o invalidada porque otro core modificó el dato.

El estado **Exclusive** permite escribir sin notificar al bus (pasa directo a Modified), reduciendo tráfico.

Ejemplos prácticos

- Predictor de ramas.
- Latencias L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ RAM.
- Multiplicación de matrices y *blocking*.

Predictor de ramas: sorted vs unsorted

Sumar los elementos mayores que un umbral:

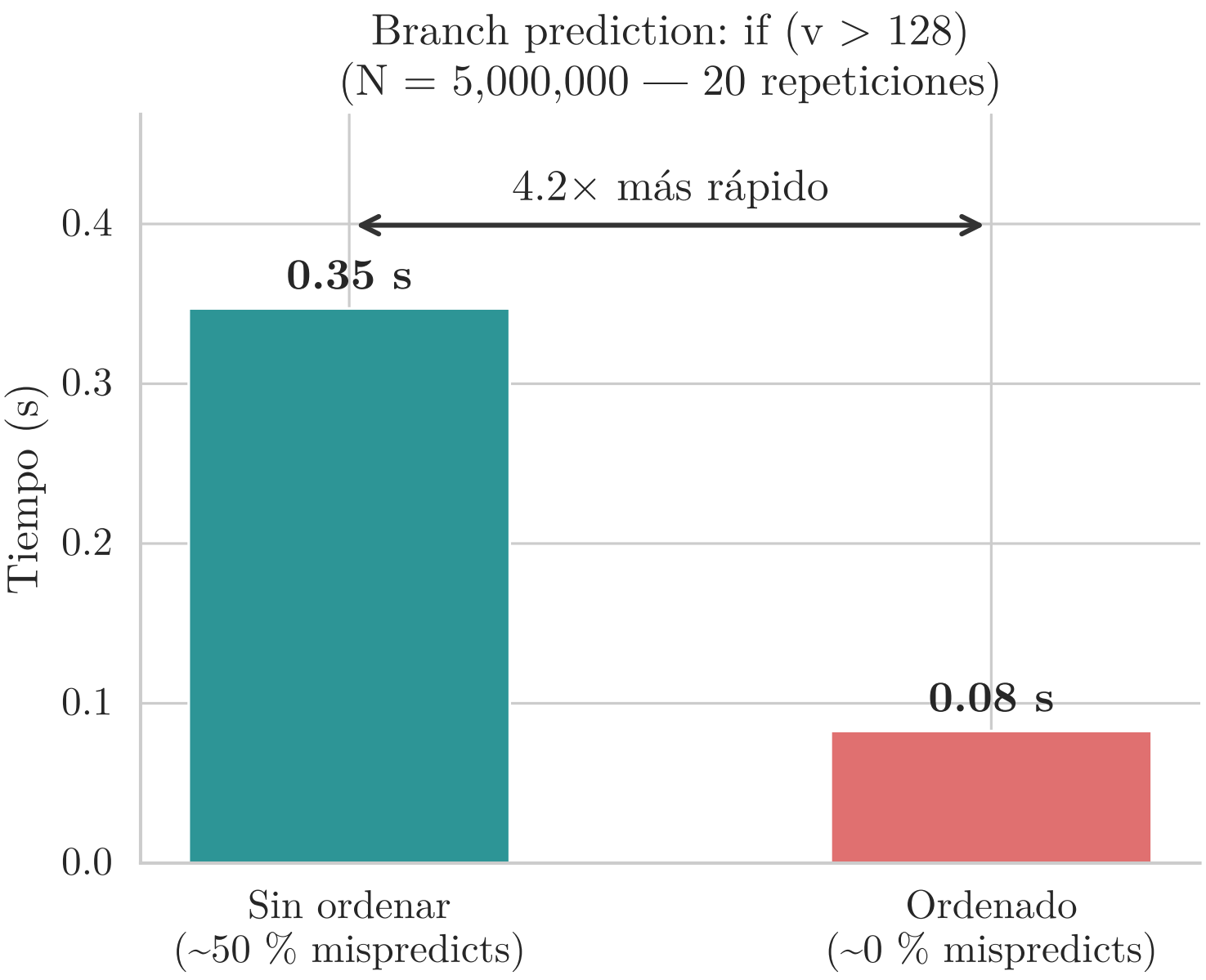
```
long long sum = 0;
for (int v : data)
    if (v > 128) sum += v;
```

- **Sin ordenar**: los valores son aleatorios en $[0, 255]$; el salto condicional es impredecible $\sim 50\,\%$ de las veces.
- **Ordenado**: primero todos los $v \leq 128$, luego todos $v > 128$.
  El predictor falla solo una vez y acierta el resto.
- El código y los datos son idénticos, solo cambia el orden.

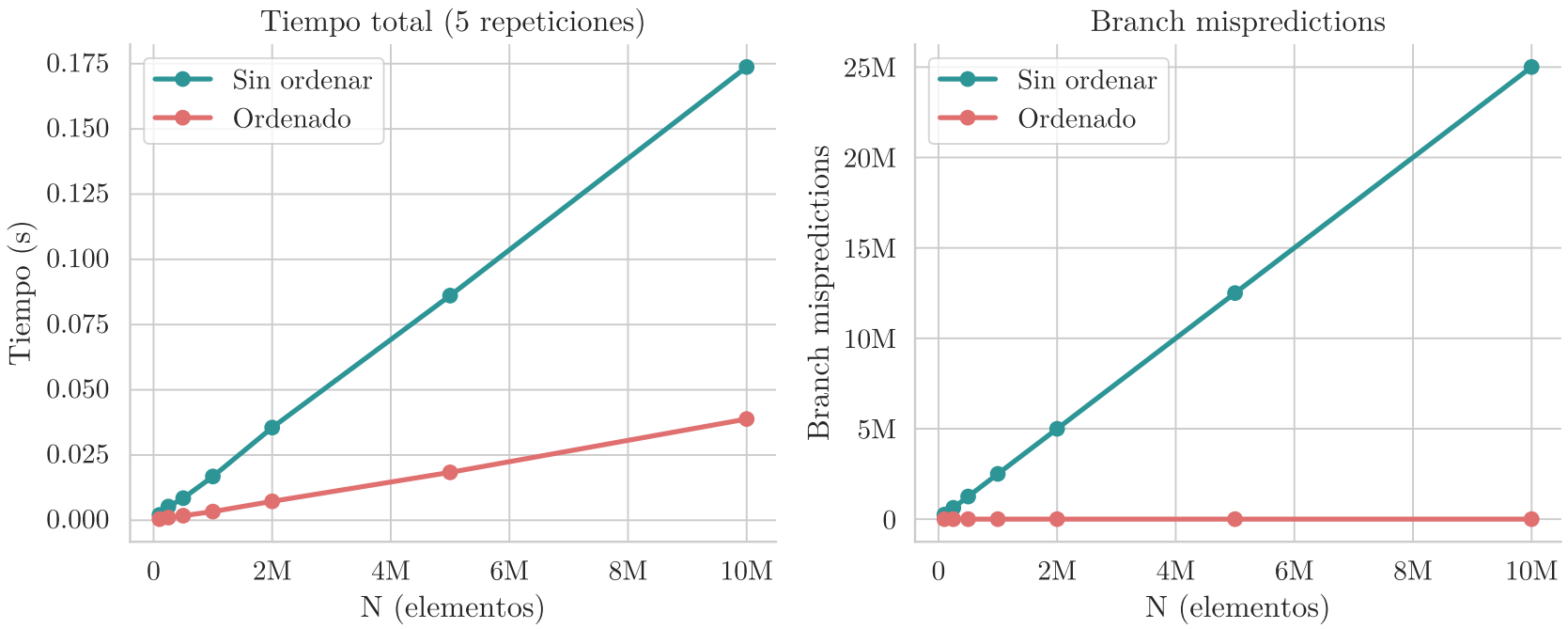
Predictor de ramas: ¿qué ocurre internamente?

- La CPU ejecuta instrucciones especulativamente : predice el resultado del salto y continúa antes de evaluarlo.
- Un **mispredict** obliga a descartar el trabajo especulativo y vaciar el pipeline ($\sim$ 15–20 ciclos de penalización).
- **Sin ordenar**: $\sim$ 50 % de mispredicts; penalización en casi todos los elementos.
- **Ordenado**: $\sim$ 0 % de mispredicts; el predictor aprende el patrón.

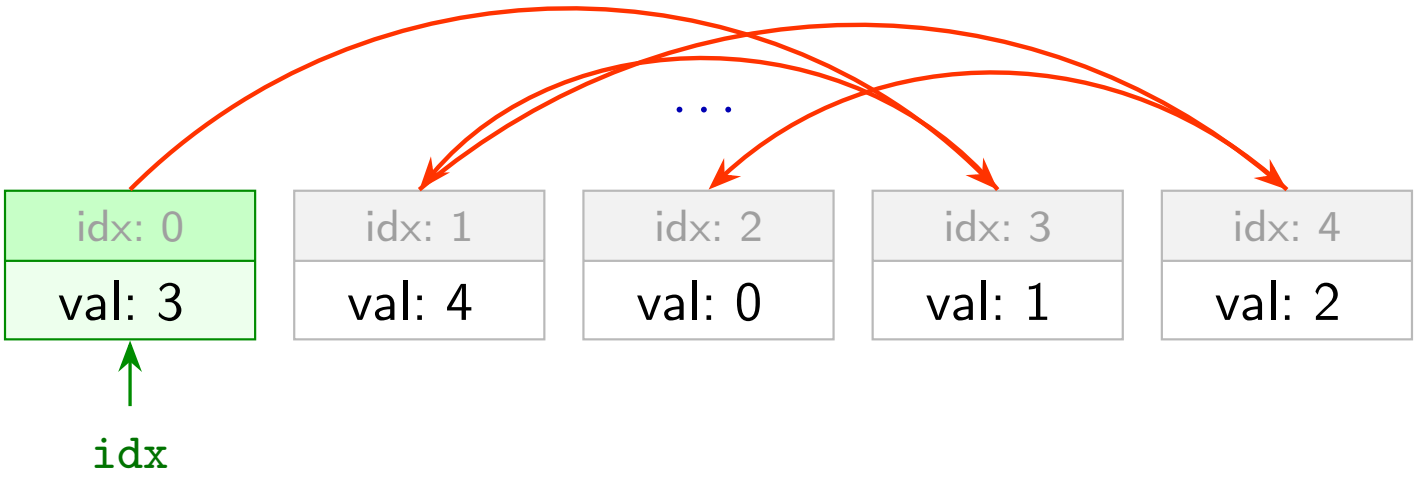
Predictor de ramas: resultado experimental



Predictor de ramas: tiempo y mispredicts vs N



Pointer chasing: recorrido impredecible



Dependencia de datos: el paso $n+1$ requiere el valor leído en el paso n

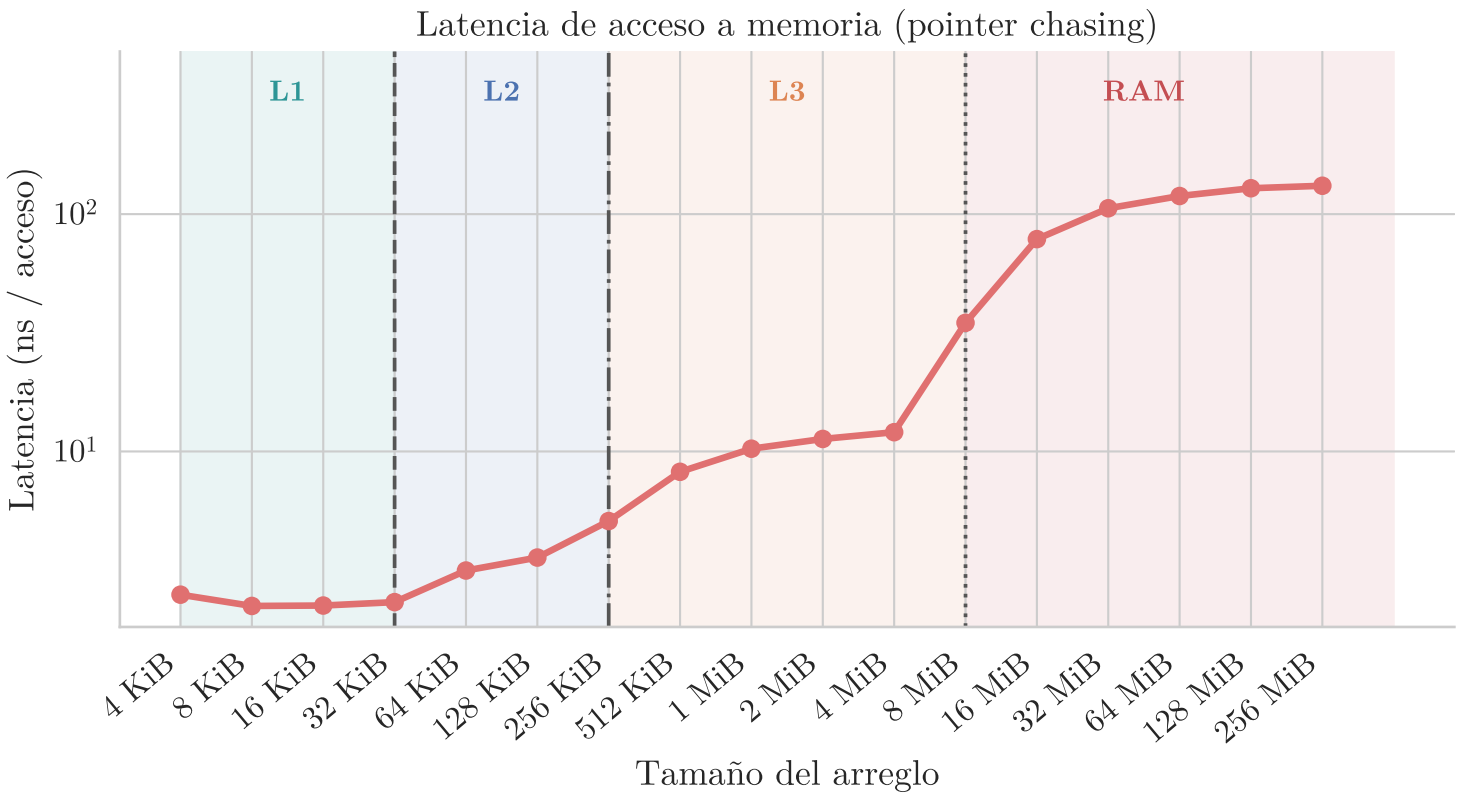
Escalera de latencias

Pointer chasing: cada posición contiene el índice de la siguiente.

```
// arr es una permutacion aleatoria de indices
for (long long s = 0; s < STEPS; s++)
    idx = arr[idx];
```

- El prefetcher no puede predecir el siguiente acceso.
- Al variar el tamaño de `arr` se revela la latencia de cada nivel.

Escalera de latencias: resultado experimental



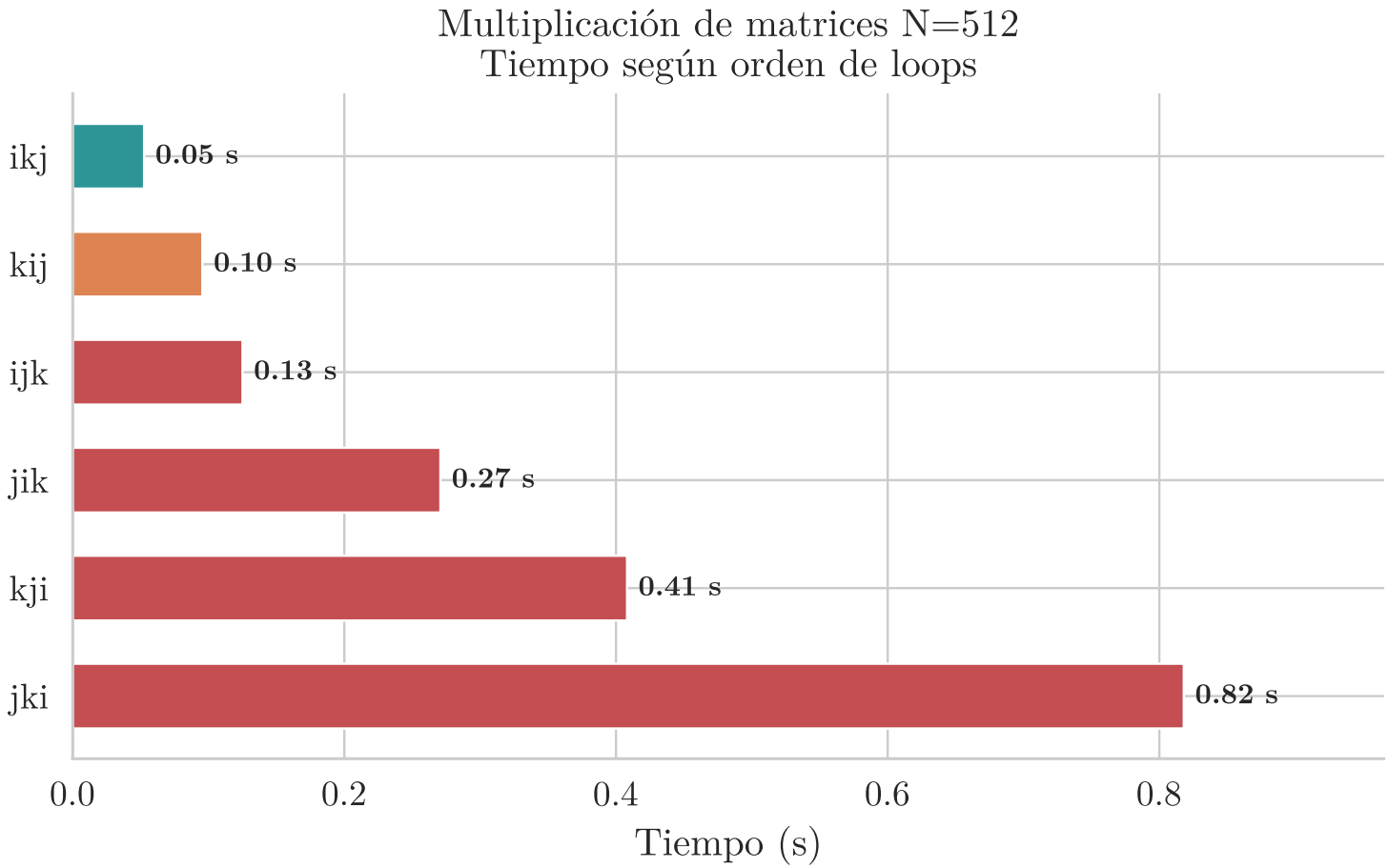
Multiplicación de matrices: el problema

La versión *naive* $C = AB$ (orden i-j-k) tiene mal rendimiento:

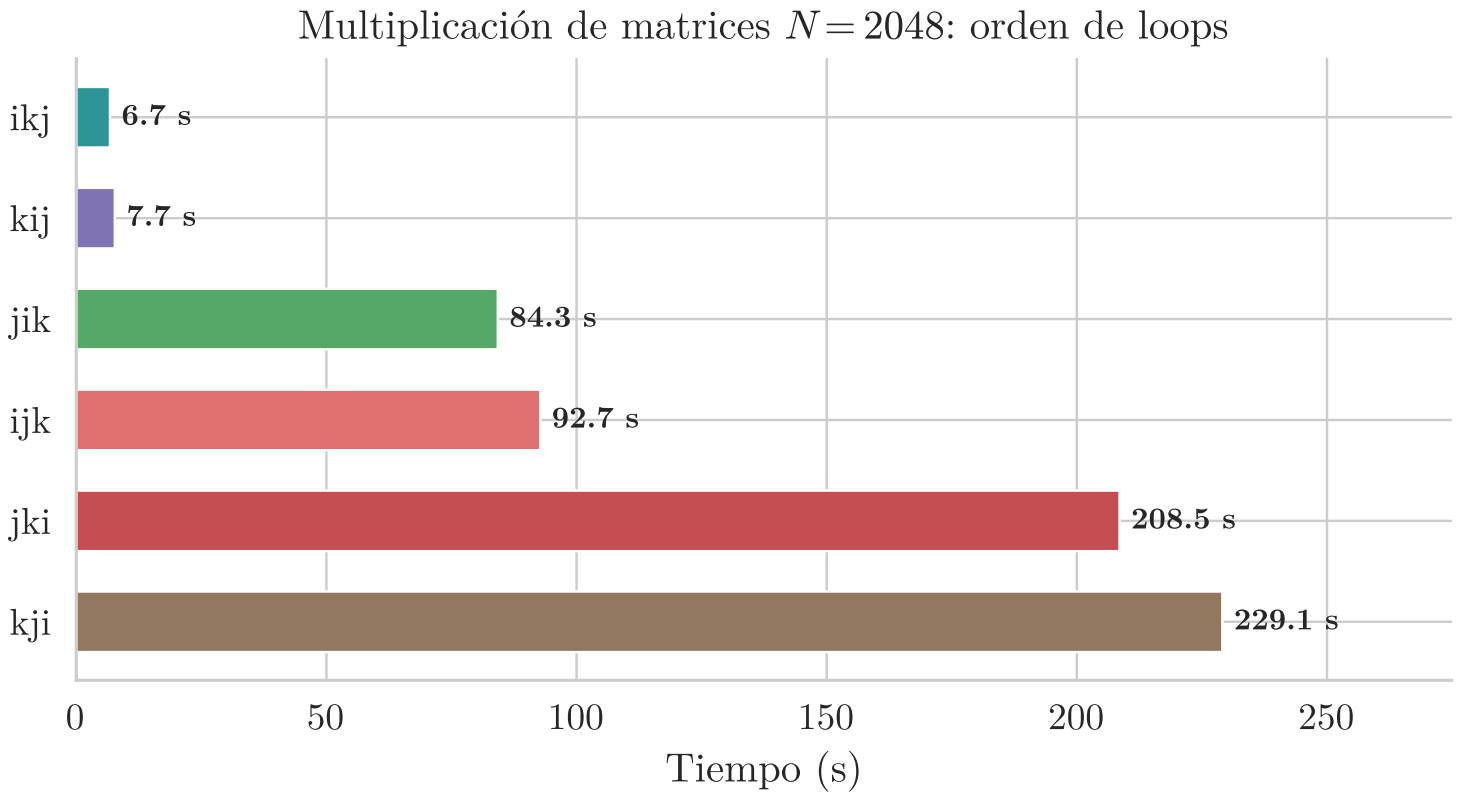
```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)
            C[i*n+j] += A[i*n+k] * B[k*n+j];
```

- El bucle interno varía k con j fijo: `B[k*n+j]` accede la **columna** j de B.

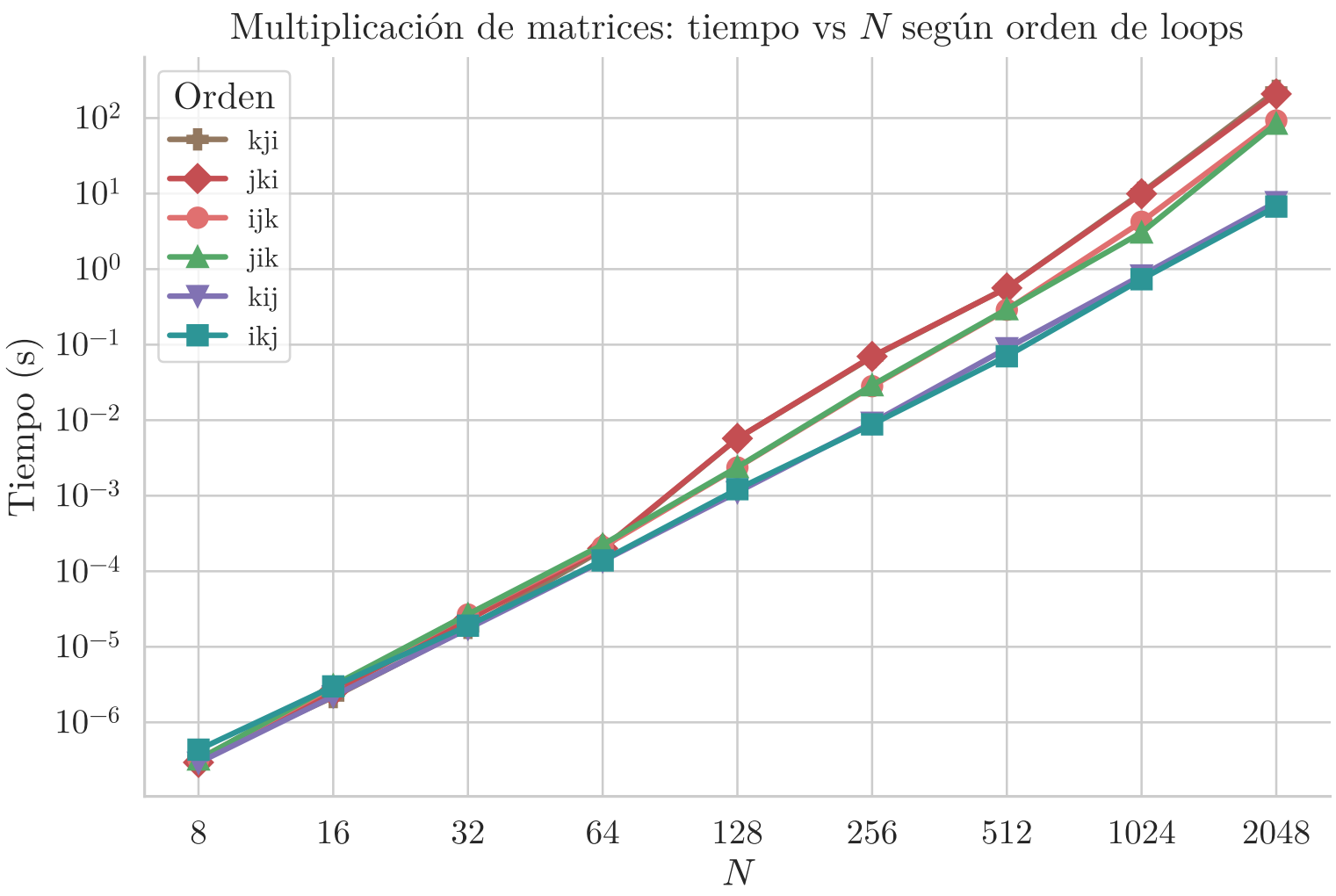
Multiplicación de matrices: impacto del orden de loops



Multiplicación de matrices: $N = 2048$



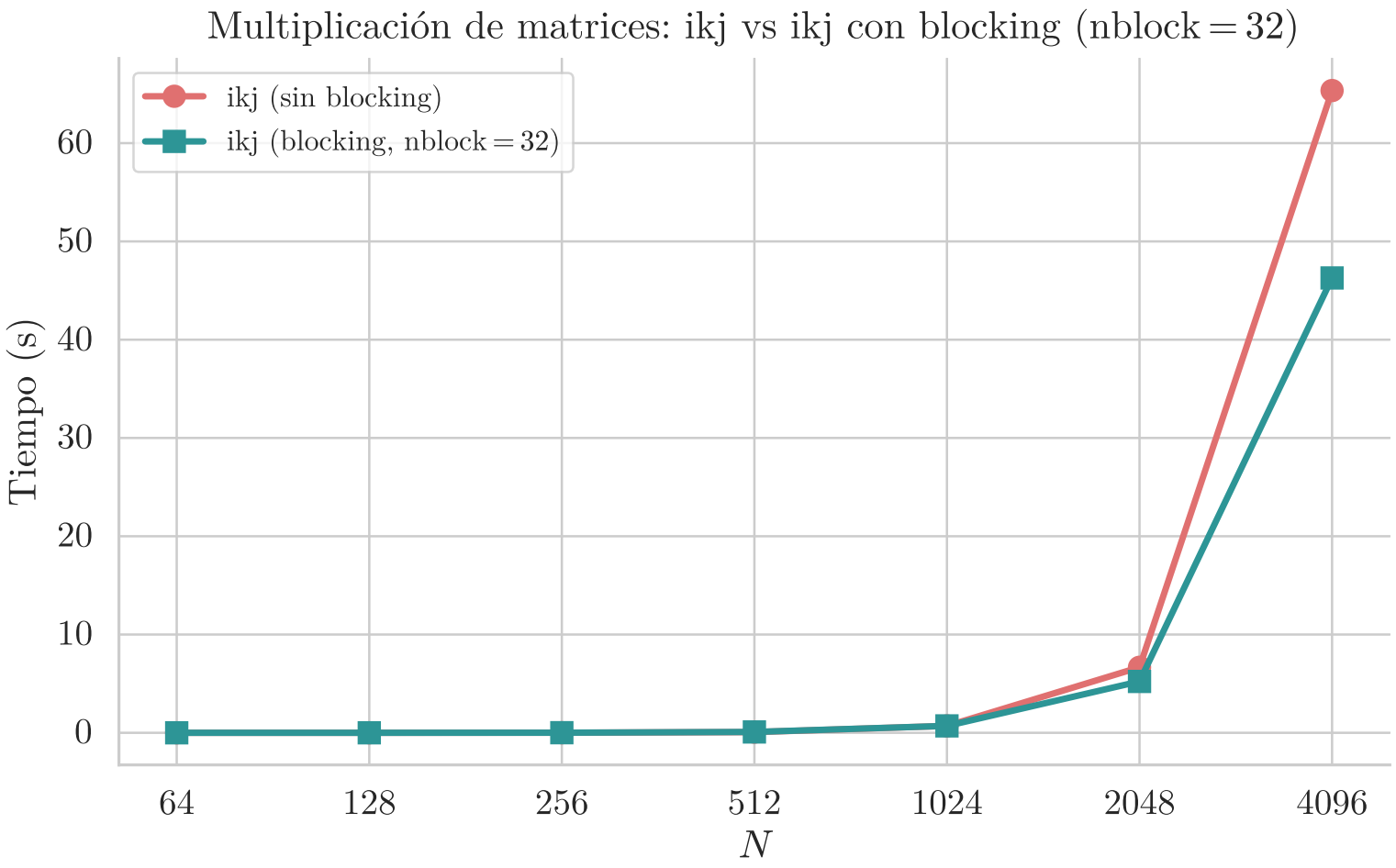
Multiplicación de matrices: escalado con N



Multiplicación de matrices: blocking

Dividimos A, B y C en bloques de `nblock` $\times$ `nblock` que caben en L1/L2:

```
for (int ii=0; ii<n; ii+=nblock)
  for (int kk=0; kk<n; kk+=nblock)
    for (int jj=0; jj<n; jj+=nblock)
      for (int i=ii; i<ii+nblock; i++)
        for (int k=kk; k<kk+nblock; k++) {
          double aik = A[i*n+k]; // reutilizado nblock veces
          for (int j=jj; j<jj+nblock; j++)
            C[i*n+j] += aik * B[k*n+j];
        }
```

- Cada bloque de B se carga una vez y se reutiliza `nblock` veces antes de desalojarse (localidad temporal).

Blocking: ikj vs ikj con blocking (`nblock` = 32)